

Goals:

- Every 5 seconds ONLY one of the following occurs (assume gap between cars) -
Simulation - Poisson Distribution (play with the numbers so it makes sense)
Research how to work with LED
 - Car enters correct parking - #1 Green
 - Car enters wrong parking - #2 Red
 - Car exits correct parking - #3 Green
 - Car exits wrong parking - #4 Blue
- Cloud Statistics - **Work with Firebase (RTDB) and ESP**
 - Aggregation + Time stamp
 - Input (state, timestamp) -> Cloud
 - Output <- how many times state #1 occurred in what time
- GUI - Show Analysis of Cars **Input: Cloud -> Output: Graphs, Pie chart**
Python application that reads from Firebase // Simple Flutter application // ...
 - Choose 3-4 kinds of analysis
 - Graph of accumulated states throughout the data
 - Pie chart - how many bad states (3-4) compared to (1-2)
 - Average number of ..., Average time for car to leave.... Average parking time
 - When car arrives, what is the probability that it takes the correct location
 - **Be creative!**

Recommended Order of Operations:

- Design of the Cloud structure
- GUI from dummy data -> Start designing the statistics etc.
- When you acquire hardware
 - Work with LEDs
 - Read & Write to Firebase

Progress Updates

Day1 14 September

- **FireBase:**
 - **Firestore Integration:** Successfully configured Firebase project with Firestore real-time database
 - **Flutter Application:** Built cross-platform mobile app with Material Design.
 - **Real-time Data Streaming:** Implemented live data synchronization between cloud and mobile interface
- **Four Event Types Implemented:**
 - Event 1: Car enters correct parking → Green LED
 - Event 2: Car enters wrong parking → Red LED
 - Event 3: Car exits correct parking → Green LED
 - Event 4: Car exits wrong parking → Blue LED
- **Mathematical Simulation:**
 - **Proper Poisson Distribution:** Implemented Knuth's algorithm for realistic event generation
 - **Configurable Lambda Parameters:**
 - Correct Entry ($\lambda=2.5$): High frequency events
 - Wrong Entry ($\lambda=0.3$): Low frequency errors
 - Correct Exit ($\lambda=1.8$): Medium frequency events
 - Wrong Exit ($\lambda=0.15$): Very rare errors
 - **5-Second Interval Simulation:** Automated event generation matching project requirements.
- **Analytics Dashboard**
 - **LED Distribution Chart:** Pie chart showing event type distribution
 - **Average Wait Time Tracking:** Novel analytics showing events between specific occurrences: The algorithm tracks how many events occur between each appearance of a specific event type. It maintains running counters and calculates averages over time.
 - **Real-time Event Feed:** Live display of recent parking events
 - **Statistics Overview:** Event counts and success rate calculations
- **User Experience**
 - **Manual Simulation Control:** Start/stop functionality via floating action button
 - **Data Management:** Clear all data feature with confirmation dialog

P Smart Parking IoT



Recent Events

26



Car exits wrong parking

CAR_871 • 22:49:18



Car enters wrong parking

CAR_484 • 22:49:13



Car enters correct parking



Car exits wrong parking

Car: CAR_871 • LED: BLUE



LED Distribution

26 Events



G

23

R

2

B

1



Avg Wait Time

Events between occurrences

1

Correct Entry

1

2

Wrong Entry

12

3

Correct Exit

3

4

Wrong Exit

26



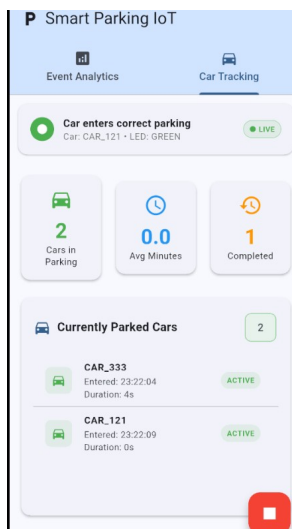
Firebase Connected

26 events stored



18 September

- **Material Design Tabs Implementation:**
 - **Tab Structure:** Split interface into two focused tabs: "Event Analytics" and "Car Tracking"
 - **UI Optimization:** Resolved screen overcrowding by distributing content across tabs
- **Car Tracking System:**
 - **Car Session Management:** Tracks individual car parking sessions with entry/exit times
 - **Active Cars Monitoring:** Real-time display of currently parked vehicles with duration tracking
 - **Session History:** Maintains record of completed parking sessions with calculated durations
 - **Statistics Cards:** Three-card summary showing Cars in Parking, Average Minutes, and Completed sessions
- **Simulation Realism Enhancement:**
 - **Smart Car ID Generation:** Implemented realistic logic where exit events preferentially use **existing** parked cars.
 - **Guaranteed Realistic Car Selection:**
 - Always select random car from existing ones for exits
 - Convert exit events to entry events when parking is empty
 - **Session Completion:** Cars can now complete full entry→exit cycles, enabling meaningful duration calculations
 - **wrongExit is a car that made a wrong entry.?? Yes**



- **Next**
 - **Hardware**
 - Planning to add more analytic statics about probability, what happens in our simulation comparing to what we expect according to the probability theory.
 - Or maybe instead of theoretical stats, I may add live park..that show visual park

19 + 20 September

Major

- Learning new skills: following the workshops guidance, I learned about:
 - Esp
 - Led, jumpers
 - tested the hardware, and wrote a program that runs the three colours serially using **serial monitor** to debug.
- Switch to **RTDB(real time database)** instead of cloud firestore

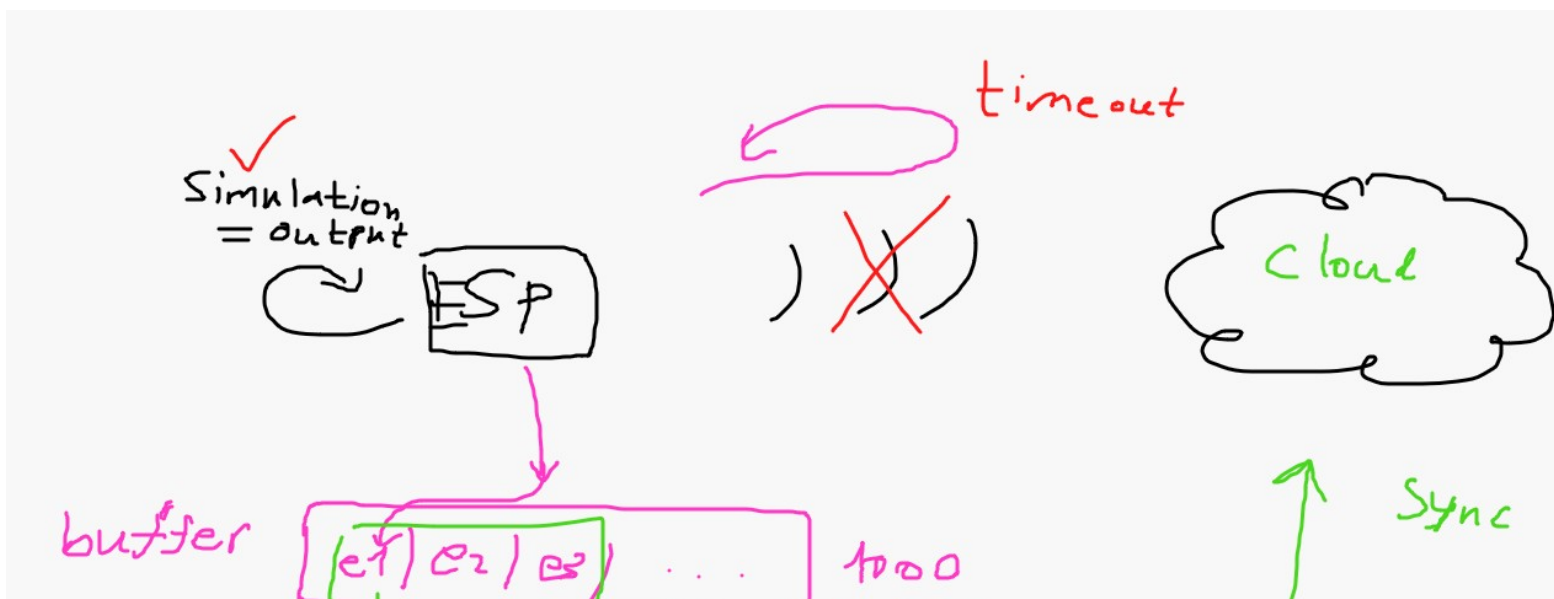
- In red = todo green- Almost Done
- Upload full ESP32 code (parking simulation)
- Connect: ESP32 → Firebase → Flutter
- Simulation Logic: same as previous one, but with C++ in ESP instead of flutter.

summary :ESP32 generates parking events every 5 seconds, sends to Firebase, Flutter shows real-time dashboard.

Almost Done with our first Goals!! Should fix some minor problems

least important

- **Flutter Changes Made:**
 - **Performance Optimization:**
 - Rebuilding relevant fields(boxes) instead of refreshing the whole page.
(**Before:** Entire screen rebuilds on every Firebase update **After:** Only specific sections rebuild when their data changes)
 - **User Interface:**
 - (By instructor advice) Event Analytics Tab: Charts → Recent Events → Control Section
 - **Removed** FAB simulation button and related animations
 - **Removed** Firebase writing methods (commented for reference)
 - **Kept** only Firebase reading functionality
 - **Added** ESP32 connection status indicator
 - **Updated** UI text to show "Waiting for ESP32 data..."
 - sorting recent data: new cars at the top of the lists for instance: recent parking sessions.
 - every 5 seconds the proper colour should be on the led, what if it is the same colour? then should blink
 - **Performance limits** - Max 20 items processed in lists



21 + 22 September

The two major changes:

- Using WiFiManager instead of hardcoding credentials.
The ESP32 becomes a temporary WiFi hotspot where users can select and configure WiFi settings through a web interface.

Handling an edge case where the **network cuts off**.
Solved by saving the data temporarily to a **buffer**, and after getting back the network upload to firebase with
!!proper **timestamps**
המטרות החדשות:

1. זיהוי ניתוק רשת - ה-ESP32 יזהה כשהאינטרנט נותק
2. **Buffer** פנימי - ישמור עד 1000 אירועים כשאין אינטרנט
3. סנכרון אוטומטי - כשהאינטרנט חוזר, ישלח את כל ה-Buffer ל-Firebase
4. **Timestamp** נכון - ישמור את הזמן האמיתי של האירוע (לא זמן קבלה ב-Firebase)

-

Minor bugs:

- List now shows all events (not limited to 20)

- Fixed avg minutes in car tracking
- Update the average minutes calculation to also count ongoing parked cars.
- Make the list of recent parking sessions sorted by exit time (descending).
- Fixed minor bug in avg wait time..

New features!:

- New graph that shows number of parked cars related to time. It updates every 30 seconds (6 events): the chart shows the **last 25 minutes** of parking occupancy history. It has **50 individual measurements** (one every 30 seconds)



-

-

```
// Poisson parameters
const double LAMBDA_CORRECT_ENTRY = 2.5;
const double LAMBDA_WRONG_ENTRY = 0.6;
const double LAMBDA_CORRECT_EXIT = 1.8;
const double LAMBDA_WRONG_EXIT = 0.9; //if no wrong parked cars, it will make it wrong
```

Updated to see more wrong entries and exits.

23 September goals:

- ☐ 1- bug car statistics: is not updating after sync ☆
- ☐ 2- make sure it works on a real phone. Try it on the tablet on your phone ☆
- ☐ 3- submit by Google drive a zip file that contains the main and the esp code and send a mail to ido that U have submitted with the link ☆
- ☐ 4- record a video to show that everything works and submit it also by drive. ☆
- ☐ 5- 20/10 your final presentation ☆
- ☐ 6-29/10 the final exam ☆

Smart Parking IoT System - Project Highlights

Core System Architecture

Complete IoT Pipeline: ESP32 → Firebase RTDB → Flutter Mobile App with real-time synchronization

Key Technical Achievements

Mathematical Simulation (Day 1)

- Proper Poisson Distribution using Knuth's algorithm
- Realistic Event Generation with configurable lambda parameters:
 - Correct Entry ($\lambda=2.5$) | Wrong Entry ($\lambda=0.3$)
 - Correct Exit ($\lambda=1.8$) | Wrong Exit ($\lambda=0.15$)
- 5-second interval automation matching project requirements

Advanced Car Tracking System (Day 4)

- Smart Car Session Management with entry/exit time tracking
- Realistic Simulation Logic: Exit events use existing parked cars only
- Guaranteed Session Completion: Full entry→exit cycles with duration calculations
- Active Cars Monitoring: Real-time display of currently parked vehicles

Mobile Analytics Dashboard

Material Design Implementation:

- Tabbed Interface: "Event Analytics" and "Car Tracking"
- Real-time Data Visualization: Pie charts, timeline graphs, event feeds
- Novel Analytics Algorithm: Tracks events between specific occurrences
- Performance Optimization: Selective rebuilding instead of full screen refresh

Hardware Integration (Day 6-7)

- ESP32 with LED System: Red/Green/Blue event indication
- WiFiManager Implementation: Web-based WiFi configuration
- Network Resilience: 1000-event buffer for offline operation
- Smart LED Control: Blinking for consecutive same-color events

Advanced Features (Day 8-9)

Network Fault Tolerance:

- Automatic network disconnection detection
- Internal buffer storing up to 1000 events during outages
- Auto-sync when connectivity restored with proper timestamps

Live Analytics:

- Parking Occupancy Graph: 25-minute history with 50 measurements
- Real-time Statistics: Average parking duration, success rates
- Session History: Complete parking records with calculated durations

Innovation Points

- ✓ Realistic Simulation: Prevents impossible scenarios (exits without entries)
- ✓ Network Resilience: Offline operation with automatic sync
- ✓ Smart Car Management: Session-based tracking with realistic behavior
- ✓ Performance Optimization: Selective UI updates for smooth operation
- ✓ Professional UX: Material Design with intuitive navigation

Technical Stack Mastery

- Embedded Programming: ESP32, WiFiManager, LED control
- Cloud Integration: Firebase RTDB with real-time synchronization
- Mobile Development: Flutter with Material Design
- Mathematical Modeling: Poisson distribution implementation
- System Architecture: Complete IoT pipeline design

Results Summary

Fully Functional System: Hardware simulation → Cloud storage → Mobile analytics with professional-grade features including network resilience, realistic car tracking, and comprehensive analytics dashboard.